AUTONOMOUS AGENT EXECUTION SPECIFICATION • PROTOCOL VER. 1.0

Project Clutch (ক্লাচ): Automated Implementation Directive

Complete engineering specification, state machines, edge-case algorithms, and sprint execution gates

Target Engine: Python (FastAPI) + Redis + PostgreSQL + Next.js	**Execution Model:** Solo Developer / "Hit-Next" Autonomous Protocol	**Security Profile:** Zero-Trust HMAC, AES-256 GCM, Strict Anti-Tamper

AUTONOMOUS "HIT-NEXT" EXECUTION DIRECTIVE: This document contains the full architecture partitioned into 5 sequential phases. Each phase provides: (1) The exact **Agent Prompt** to copy-paste, (2) The **Verification Test** to run in your VS Code terminal, and (3) The exact **Git Command** to automatically stage, commit, and push your progress to GitHub before advancing to the next step.

1. SYSTEM ARCHITECTURE & COMPONENT HIERARCHY

Clutch enforces strict decoupling between external platform ingestion, state accumulation, parsing, and logistics dispatch. All external API calls that may suffer network timeouts are non-blocking and isolated behind distributed queues.

```
      # Component Hierarchy & Event Flow [Meta Graph Webhooks] --> [FastAPI Ingestion] --> [Redis Debounce Buffer (75s TTL)] | (TTL Expire Trigger) v [Core Normalizer & Parser Engine] - Bengali Unicode Digit Converter - Phone Sanitizer (013-019 MSISDN) - 3-Tier Zone & Thana Resolver | v [PostgreSQL Database (State: PENDING)] | [Next.js Mobile Console] <-- (WebSocket / Poll) ------+ | v (Tap: "⚡ DISPATCH TO STEADFAST") [Atomic Redis Lock] --> [Celery/RQ Queue Worker] --> [Steadfast/Pathao API] | (Success: Consignment SF-XXXXXX) v [Telegram Bot Merchant Broadcast]
```

2. NORMALIZED RELATIONAL SCHEMA (POSTGRESQL)

The persistent layer must track order state transitions, unique consignments, and fraud signatures with strict idempotency keys.

TABLE	KEY FIELDS & TYPES	INDEXES, CONSTRAINTS & BUSINESS RULES
`merchants`	`id` (UUID), `page_id` (VARCHAR), `access_token` (TEXT, ENC), `sf_api_key` (TEXT)	Unique index on `page_id`. Token health monitor validates token validity hourly.
`orders`	`id` (UUID), `merchant_id`, `sender_id`, `raw_text`, `clean_phone` (VARCHAR 11), `address`, `zone`, `cod_amount` (DECIMAL), `status`	Unique composite index: `(merchant_id, sender_id, created_at::date)`. Status: `pending`, `needs_review`, `queued`, `dispatched`, `cancelled`.
`consignments`	`id`, `order_id`, `courier` (ENUM), `tracking_code`, `consignment_id`, `pickup_status`	Unique index on `consignment_id`. Foreign key cascade to `orders.id`. Prevents duplicate booking.
`trx_audit_log`	`id`, `trx_id` (VARCHAR 16), `provider` (bKash/Nagad), `order_id`, `verified_at`	Unique constraint on `trx_id`. Blocks duplicate TrxID reuse fraud across all merchant accounts.

3. MANDATORY ALGORITHMIC GUARDRAILS (AGENT CODE TASKS)

The autonomous agent must implement these exact algorithmic functions within the core ingestion worker:

A. Multi-Message Debounce Buffer (Redis)

When a customer sends short message bursts, append incoming payloads to a list: `session:{page_id}:{sender_id}`. Reset TTL to 75 seconds. Once the TTL expires with no new incoming messages, pop all items, concatenate into a single text block, and pass to the parser.

B. Bengali Unicode Digit Normalizer & Phone Sanitizer

```
def normalize_bd_phone(raw_text: str) -> list[str]: # 1. Map Bengali digits (০-৯) to ASCII (0-9) bn_map = str.maketrans("০১২৩৪৫৬৭৮৯", "0123456789") ascii_text = raw_text.translate(bn_map) # 2. Extract 11-digit numbers with BD prefixes (013-019) sanitized = re.sub(r"[\s\-\+\(\)]", "", ascii_text) matches = re.findall(r"(?:8801|01)[3-9]\d{8}", sanitized) return [m[-11:] for m in matches] # Normalize to standard 11 digits
```

C. 3-Tier Delivery Zone & Suburb Classifier

Evaluate raw text against geographic dictionaries before assigning courier charges:

- **Inside Dhaka (৳70):** Match core zones (Uttara, Mirpur, Dhanmondi, Gulshan, Mohammadpur, Badda, Banani, Motijheel, etc.).
- **Dhaka Suburbs (৳100):** Match boundary towns: *Savar, Gazipur, Tongi, Keraniganj, Narayanganj, Ashulia*.
- **Outside Dhaka (৳130):** Match any other of the 64 districts or default fallback.

D. Human Takeover Suppression Sensor

Intercept Meta webhook payloads where `entry.messaging.message.is_echo == True` or sender is Page Admin. Immediately execute: `redis.set(f"takeover:{page_id}:{sender_id}", 1, ex=2700)` (45-min TTL). The bot must not emit any automated response while this key exists.

E. Double-Click Dispatch Lock

Wrap courier booking in an atomic distributed lock: `lock = redis.set(f"lock:dispatch:{order_id}", "locked", nx=True, ex=30)`. If acquisition fails, abort and return `{"status": "already_processing"}` to prevent duplicate courier consignments.

4. ENTERPRISE SECURITY & ANTI-EXPLOIT ARCHITECTURE

Because Clutch handles sensitive customer PII, merchant financial balances, and Meta page management tokens, the system implements a strict multi-layer security posture:

VULNERABILITY VECTOR	ATTACK MECHANISM / THREAT	MANDATORY HARDENING & MITIGATION STANDARD
Webhook Spoofing	Malicious actor injects forged Meta comment/DM payloads to drain API quotas or push fake orders.	**HMAC-SHA256 Verification:** Validate raw payload against `X-Hub-Signature-256` header using merchant's App Secret before JSON parsing. Reject on mismatch.
Credential & Token Theft	Database dump exposes long-lived Meta Page Access Tokens and Steadfast API keys.	**AES-256-GCM Envelope Encryption:** All 3rd-party tokens stored at rest encrypted via AES-256-GCM with keys isolated in environment variables.
API Abuse & DoS	Bot floods the public webhook endpoint to exhaust server memory and trigger rate limits.	**Redis Token Bucket Rate Limiting:** Enforce strict 60 req/min limit per IP; edge-proxy Cloudflare rules blocking non-Meta IP ranges on `/webhooks`.
Injection & Script Poisoning	Attacker injects SQL payloads or XSS vectors through conversational Facebook DM fields.	**Parameterized Queries & Strict Typings:** Enforce Pydantic v2 input sanitization and SQLAlchemy prepared statements. Reject raw SQL strings entirely.
IDOR & Consignment Hijacking	Merchant A attempts to inspect or cancel orders belonging to Merchant B by guessing order UUIDs.	**Tenant Isolation:** Every query enforces a hard `WHERE merchant_id = current_user.tenant_id` clause validated via signed JWT session cookies.

5. MERCHANT CONSOLE UI TOKEN SPECIFICATIONS

The Next.js/Tailwind frontend must strictly implement the Cyber-Magenta & Deep Obsidian design system with high-contrast text and uncrowded 24px spatial margins:

- **Canvas Void:** `#0E0A12` • **Surface Slate:** `#16101D` • **Border Wire:** `#341B3B`
- **Primary Kinetic Accent:** `#FF1493` (Hot Cyber Magenta) — Reserved strictly for the single dispatch trigger.
- **Active Automation State:** `#00F5A0` (Acid Mint) — Webhook listening, API online, consignment confirmed.
- **Warning / Fraud State:** `#F59E0B` (Signal Amber) — Return risk >40%, advance payment unverified.

6. PHASED "HIT-NEXT" SEQUENTIAL EXECUTION BLUEPRINT

Execute each block in order. Feed each prompt into your AI agent or Copilot, run the verification test, and commit to GitHub using the terminal commands below before typing *"Next"*.

Phase 1: Ingestion, Normalization & Meta HMAC Security

Goal: Parse messy Bengali text into clean database phone numbers, map 3-tier delivery zones, and enforce HMAC-SHA256 signature verification.

```
# PROMPT TO COPY-PASTE FOR PHASE 1: "Build Phase 1 of Project Clutch in `app/core/sanitizer.py` and `app/core/security.py`. Implement: 1. `normalize_bd_phone(text)` converting Bengali numerals (০-৯) to ASCII digits and extracting 11-digit numbers (013-019). 2. `resolve_zone(address)` classifying addresses into Inside Dhaka (৳70), Suburbs (৳100), or Outside Dhaka (৳130). 3. `verify_meta_signature(payload_bytes, signature_header, secret)` validating X-Hub-Signature-256 using HMAC-SHA256. Write complete, working unit tests in `tests/test_phase1.py` with 10 dirty Banglish strings. Zero placeholders."
```

```
# TERMINAL VERIFICATION & GIT PUSH: python -m pytest tests/test_phase1.py -v git add . git commit -m "feat(phase-1): core sanitizer, 3-tier zone engine, and meta hmac security" git push
```

Phase 2: Redis Debounce Buffer & Admin Takeover Guard

Goal: Buffer split multi-message bursts across 75 seconds and mute the bot for 45 minutes whenever a human admin manually replies.

```
# PROMPT TO COPY-PASTE FOR PHASE 2: "Phase 1 is committed. Build Phase 2 in `app/services/session_buffer.py` and `app/api/webhooks.py`. Implement: 1. Redis-backed accumulator appending text to `session:{sender_id}` with a 75-second sliding TTL. 2. An Admin Presence Sensor that listens for `is_echo: true` or admin messages and sets `admin_active:{sender_id}` in Redis with a 45-minute TTL to suppress bot replies. 3. A 24-hour Meta messaging window timer rejecting auto-messages after 23h 50m. Provide unit tests in `tests/test_phase2.py` simulating chat bursts and human admin takeovers."
```

```
# TERMINAL VERIFICATION & GIT PUSH: python -m pytest tests/test_phase2.py -v git add . git commit -m "feat(phase-2): 75s redis session debounce and 45min admin takeover sensor" git push
```

Phase 3: Database Schema & Resilient Courier Worker

Goal: Set up PostgreSQL persistence, Celery retry queues, and atomic distributed locks to prevent double parcel bookings.

```
# PROMPT TO COPY-PASTE FOR PHASE 3: "Phase 2 is committed. Build Phase 3 in `app/models/schema.py`, `app/workers/dispatch.py`, and `app/adapters/steadfast.py`. Implement: 1. PostgreSQL schema for `merchants`, `orders`, `consignments`, and `trx_audit_log` with AES-256 encrypted token fields. 2. Atomic Redis lock (`lock:dispatch:{order_id}`) preventing double-click race conditions. 3. Steadfast API client with offline Thana/District mapping and exponential backoff retries (30s, 2m, 10m) for HTTP 500/502/504 errors. Include test script `tests/test_phase3.py` verifying retry queues during simulated gateway outages."
```

```
# TERMINAL VERIFICATION & GIT PUSH: python -m pytest tests/test_phase3.py -v git add . git commit -m "feat(phase-3): postgres schema, steadfast adapter, and atomic dispatch lock" git push
```

Phase 4: Cyber-Magenta Mobile Merchant Console

Goal: Build an uncrowded, high-contrast mobile dashboard in Next.js 14 and Tailwind CSS for single-touch order dispatching.

```
# PROMPT TO COPY-PASTE FOR PHASE 4: "Backend pipeline is complete. Build Phase 4 in `frontend/`: 1. Implement Next.js 14 mobile console using Tailwind tokens: Canvas `#0E0A12`, Surface `#16101D`, Accent `#FF1493`, and Live Status `#00F5A0`. 2. Create order review cards showing side-by-side Raw Customer DM vs Sanitized Fields (Phone, Address, Zone, COD). 3. Provide 1-tap inline editing for address corrections and a multi-state button: '⚡ DISPATCH TO STEADFAST' -> Processing -> Booked (SF-XXXXXX). 4. Add operational pickup badge: '🟢 Same-Day Pickup' (<3:30 PM) vs '🟠 Next-Day Morning Dispatch'."
```

```
# TERMINAL VERIFICATION & GIT PUSH: npm --prefix frontend test git add . git commit -m "feat(phase-4): cyber-magenta mobile console and single-tap dispatch feed" git push
```

Phase 5: Fraud Shields, Telegram Alerts & Pilot Test

Goal: Block fake bKash TrxIDs, flag serial returners, and push real-time booking alerts to Telegram.

```
# PROMPT TO COPY-PASTE FOR PHASE 5: "Build Phase 5: 1. `app/services/fraud.py`: Validate 10-char bKash TrxIDs against `trx_audit_log` to prevent reuse. Query courier API delivery ratios and flag buyers with <60% success rate as amber 'High Return Risk'. 2. `app/services/telegram_bot.py`: Broadcast instant parcel booking alerts with tracking URLs. 3. `scripts/chaos_test.py`: Run an automated test sending 50 concurrent messy orders with simulated 504 courier drops. Verify zero dropped parcels and zero unhandled exceptions."
```

```
# TERMINAL VERIFICATION & GIT PUSH: python scripts/chaos_test.py git add . git commit -m "feat(phase-5): fraud protection, telegram alerts, and chaos verification" git push
```